

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г. В. Плеханова»
МОСКОВСКИЙ ПРИБОРОСТРОИТЕЛЬНЫЙ ТЕХНИКУМ

Специальность: 09.02.07 Информационные системы и программирование
Квалификация: Программист

ПРАКТИЧЕСКАЯ РАБОТА №2
ПО ОСНОВАМ АЛГОРИТМИЗАЦИИ И
ПРОГРАММИРОВАНИЯ
ТЕМА: Текстовая игра-новелла
Листов: 9

Выполнил студент
Группы П1-24
Андреева Н Д

Проверил преподаватель
А А Никонова
« ____ » _____ 2025 года

Москва 2025

Цель работы Разработать текстовую игру, используя основы программирования на Python, с упором на работу с коллекциями данных (списки, кортежи, множества, словари).

Ход работы

1. Импорт библиотек `random` и `time`, функция `slow_print`

`import random` используется для случайного выбора преступника и ответов подозреваемых. `import time` - позволяет делать паузы между сообщениями. Функция `slow_print` замедляет появление текста на экране. `for char in text:` перебирает каждый символ в строке `text`. `print(char, end="", flush=True)` - выводит символ без переноса строки. `time.sleep(delay)` —делает паузу между символами. `print()` в конце добавляет перевод строки после вывода всей фразы

```
1  import random
2  import time
3
4  def slow_print (text, delay=0.05):
5      for char in text:
6          print(char, end="", flush=True)
7          time.sleep(delay)
8      print()
```

Рисунок 1 - Импорт библиотек `random` и `time`, функция `slow_print`

2. Приветствие

Выводится приветственное сообщение с помощью функции `slow_print`. `time.sleep(2)` - пауза на 2 секунды `suspects` - это список из пяти подозреваемых. `culprit` — случайный выбор одного из подозреваемых с помощью функции `random.choice()`. `found_clues` —

пустой список, в который будут добавляться найденные улики
evidence_points - очки доказательств.

```
9
10 slow_print("Добро пожаловать в игру 'Детективное расследование'!")
11 print("Ты – детектив, расследующий загадочную кражу из музея.")
12 print("Твоя задача – собрать улики, допросить подозреваемых и найти преступника!\n")
13
14 time.sleep(2)
15
16
17 suspects = ["директор", "охранник", "гид", "уборщица", "реставратор"]
18 culprit = random.choice(suspects)
19
20 found_clues = []
21 evidence_points = 0
```

Рисунок 2 - Приветствие

3. Уровень 1. Осмотр места преступления.

Игроку предлагается меню с выбором мест для осмотра. while
True - бесконечный цикл, который работает до break

```
23 print("=== УРОВЕНЬ 1: ОСМОТР МЕСТА ПРЕСТУПЛЕНИЯ ===")
24 slow_print("Тыходишь в музей. Витрина, где хранился бриллиант, пуста.")
25 slow_print("Выбери, с чего начнёшь осмотр.\n")
26
27 searched_rooms = []
28
29 while True:
30     print("Доступные места осмотра:")
31     print("1 – Витрина")
32     print("2 – Кабинет директора")
33     print("3 – Комната охраны")
34     print("4 – Коридор")
35     print("5 – Завершить осмотр")
36     choice = input("Твой выбор: ")
```

Рисунок 3 - Уровень 1. Цикл while True

if choice == "1": —проверяет, что именно выбрал игрок и если
данный выбор еще не находится в списке searched_rooms, то
программа выводит текст. Вслучае если этот выбор есть в списке или
произведен неправильный ввод программа выводит: «Это место уже

осмотрено или неправильный выбор». `found_clues.append("")` - сохраняет найденные улики `search_rooms.append` - помечает место как осмотренное и добавляет его в соответствующий список. `evidence_points += «»` - увеличивает очки пользователя. Каждое место осмотра даёт разные улики и очки. `/n` - переносит строку. `.extend` - добавляет несколько элементов сразу.

```
38     if choice == "1" and "витрина" not in searched_rooms:
39         print("\nТы осматриваешь витрину...")
40         print("На стекле – аккуратный разрез, как будто кто-то использовал специальный инструмент.\n")
41         time.sleep(1)
42         found_clues.append("след от инструмента")
43         evidence_points += 1
44         searched_rooms.append("витрина")
45     elif choice == "2" and "кабинет директора" not in searched_rooms:
46         print("\nТы осматриваешь кабинет директора...")
47         print("В кабинете директора беспорядок. На полу валяется ключ-карта и квитанция из ломбарда.\n")
48         time.sleep(1)
49         found_clues.extend(["ключ-карта", "квитанция"])
50         evidence_points += 2
51         searched_rooms.append("кабинет директора")
52     elif choice == "3" and "комната охраны" not in searched_rooms:
53         print("\nТы осматриваешь комнату охраны...")
54         print("Ты находишь журнал дежурств. Камеры были выключены с 23:00 до 23:10 – именно во время кражи.\n")
55         time.sleep(1)
56         found_clues.append("журнал дежурств")
57         evidence_points += 1
58         searched_rooms.append("комната охраны")
59     elif choice == "4" and "коридор" not in searched_rooms:
60         print("\nТы осматриваешь коридор...")
61         print("На полу грязные следы, а рядом кусочек ткани от формы.\n")
62         time.sleep(1)
63         found_clues.append("кусочек ткани")
64         evidence_points += 1
65         searched_rooms.append("коридор")
66     elif choice == "5":
67         if len(searched_rooms) == 0:
68             print("Ты пока ничего не осмотрел. Начни хотя бы с одного места.\n")
69             time.sleep(1)
70         else:
71             print("\nТы завершил осмотр музея. Пора переходить к анализу улик.\n")
72             break
73     else:
74         print("Это место уже осмотрено или неправильный выбор.\n")
75         time.sleep(1)
76
77 time.sleep(2)
```

Рисунок 4 - Уровень 1. Осмотр места преступления.

4. Уровень 2 Анализ улик

После осмотра мест преступления программа рассматривает все найденные улики при помощи цикла for.

for clue in found_clues: — берёт каждую улику из списка. Условие if проверяет, какая именно улика. В зависимости от найденных улик добавляются очки. Также программа выводит человека, к кому улика может относиться. Если улики не найдены программа выводит: «Улики не найдены».

```
79 print("=== УРОВЕНЬ 2: АНАЛИЗ УЛИК ===")
80 slow_print("Теперь нужно понять, кому принадлежат найденные предметы.\n")
81
82 time.sleep(2)
83
84 for clue in found_clues:
85     print(f"- Улика: {clue}")
86     time.sleep(2)
87     if clue == "след от инструмента":
88         print("> Похоже, преступник использовал профессиональные инструменты. Возможно, реставратор.")
89         evidence_points += 1
90     elif clue == "ключ-карта":
91         print("> Доступ к музею ночью есть только у директора и охраны.")
92         evidence_points += 2
93     elif clue == "журнал дежурств":
94         print("> Камеры отключались вручную. Охранник может быть причастен.")
95         evidence_points += 2
96     elif clue == "кусоч ткани":
97         print("> Ткань напоминает униформу уборщицы.")
98         evidence_points += 1
99     elif clue == "квитанция":
100         print("> Кто-то закладывал дорогие украшения в ломбарде... подозрительно.")
101         evidence_points += 2
102     else:
103         print("> Пока непонятно, кому это может принадлежать.")
104
105 time.sleep(2)
106
107 slow_print("\nВы закончил анализ. Теперь можно задать уточняющие вопросы свидетелям.\n")
108
109 time.sleep(2)
```

Рисунок 5 - Уровень 2 Анализ улик

5. Уровень 3. Допрос подозреваемых.

`for i in range(3)` - позволяет осуществить допрос 3 раза.
`print("Подозреваемые:", " ".join(suspects))` - выводит список подозреваемых через запятую
`input(" ").lower()` - приводит ввод к нижнему регистру.

```

111 print("=== УРОВЕНЬ 3: ДОПРОС ПОДОЗРЕВАЕМЫХ ===")
112 slow_print("Ты можешь допросить только трёх человек! Задавай им вопросы.\n")
113
114 time.sleep(0.5)
115
116 for i in range(3):
117     print("Подозреваемые:", " ".join(suspects))
118     person = input("Кого допросим? ").lower()
119

```

Рисунок 6 - Уровень 3 `for i in range(3)`

Если введенного человека нет в списке, выводится: «Такого человека нет.». Далее программа позволяет пользователю ввести вопрос для подозреваемого и случайно выбирает его реплику. Если он виновен — его ответы звучат подозрительно, если нет - это обычные ответы. `.capitalize` делает первую букву заглавной в выводе. Переменная `cont` позволяет закончить допрос раньше.

```

120 if person not in suspects:
121     print("Такого человека нет.")
122     continue
123
124 print(f"\nТы допросил {person}.")
125 time.sleep(1)
126 slow_print("Задай вопрос (например: 'Где вы были ночью?' или 'Чем вы занимались этой ночью?'):")
127 question = input("Вопрос: ")
128 time.sleep(1)
129
130 print(f"{person.capitalize()} отвечает...")
131 time.sleep(2)
132
133 if person == culprit:
134     response = random.choice([
135         "Ну... я не обязан на это отвечать.",
136         "Я был дома... один.",
137         "Это всё совпадение!",
138         "Вы меня подозреваете зря!"
139     ])
140 else:
141     response = random.choice([
142         "Я ничего не знаю.",
143         "Был на рабочем месте, можете проверить.",
144         "Слышал, как сигнализация сработала.",
145         "Всё это ужасно!"
146     ])
147 print(response)
148
149 if person == culprit:
150     print("Ты чувствуешь, что он нервничает... что-то здесь не так.")
151     evidence_points += 3
152 else:
153     print("Похоже, он говорит правду.")
154     evidence_points += 1
155
156 cont = input("Продолжить допрос? (да/нет): ").lower()
157 if cont == "нет":
158     break
159

```

Рисунок 7 - Уровень 3. Допрос подозреваемых.

6. Уровень 4. Дополнительный поиск

Это дополнительная возможность поиска улики для внимательных игроков. Можно найти ещё улики, которые помогут подтвердить вину. Новый цикл `while True` создаёт возможность искать несколько раз. В зависимости от выбора добавляются разные улики.

```
161 print("\n== УРОВЕНЬ 4: ВОЗВРАТ НА МЕСТО ПРЕСТУПЛЕНИЯ ==")
162 slow_print("Ты решаешь проверить музей ещё раз – вдруг осталась незамеченная улика.\n")
163
164 time.sleep(2)
165
166 while True:
167     print("Где искать?")
168     print("1 – Под витриной")
169     print("2 – У задней двери")
170     print("3 – В подсобке уборщицы")
171     print("4 – Завершить поиск")
172
173     extra_choice = input("Твой выбор: ")
174
175     if extra_choice == "1":
176         print("\nПод витриной ты находишь крошечный кусочек стекла с отпечатком пальца.")
177         found_clues.append("отпечаток пальца")
178         evidence_points += 3
179     elif extra_choice == "2":
180         print("\nУ двери найден ботинок с грязью – но не музейной.")
181         found_clues.append("ботинок")
182         evidence_points += 2
183     elif extra_choice == "3":
184         print("\nВ подсобке стоит ведро, а внутри – тряпка с песком от витрины.")
185         found_clues.append("тряпка с песком")
186         evidence_points += 3
187     elif extra_choice == "4":
188         print("Ты завершил дополнительный поиск.")
189         break
190     else:
191         print("Нет такого варианта.")
192     time.sleep(1)
193     cont = input("Искать дальше? (да/нет): ").lower()
194     if cont == "нет":
195         break
196
197 time.sleep(2)
```

Рисунок 8 - Уровень 4. Дополнительный поиск

7. Уровень 5, 6. Сопоставление фактов и обвинение

На этом этапе все улики сравниваются с личностью преступника. Если совпадение найдено — игрок получает дополнительные очки

```
198
199 print("\n=== УРОВЕНЬ 5: СОПОСТАВЛЕНИЕ ФАКТОВ ===")
200 slow_print("Ты собираешь все улики воедино.\n")
201
202 time.sleep(1)
203
204 for clue in found_clues:
205     if clue == "отпечаток пальца" and culprit == "директор":
206         print("Отпечаток совпадает с директором!")
207         evidence_points += 5
208     elif clue == "ботинок" and culprit == "охранник":
209         print("Размер ботинка совпадает с охранником.")
210         evidence_points += 3
211     elif clue == "тряпка с песком" and culprit == "уборщица":
212         print("Песок с витрины – уборщица явно здесь была.")
213         evidence_points += 4
214     elif clue == "след от инструмента" and culprit == "реставратор":
215         print("След совпадает с инструментом реставратора!")
216         evidence_points += 4
217
218 time.sleep(1)
219 slow_print("\nТеперь у тебя достаточно данных, чтобы обвинить преступника.\n")
220 time.sleep(1)
```

Рисунок 9 - Уровень 5. Сопоставление фактов

Игрок делает выбор на основе своих наблюдений, и программа проверяет — угадал он или нет. Также программа сверяет количество набранных баллов и выводит соответствующий результат.

```
221
222 print("=== УРОВЕНЬ 6: ОБВИНЕНИЕ ===")
223 print("Подозреваемые:", ", ".join(suspects))
224 guess = input("Кого ты считаешь виновным? ").lower()
225
226 time.sleep(1)
227
228 print("\n=== РЕЗУЛЬТАТ ===")
229 if guess == culprit:
230     slow_print(f"Верно! Преступником оказался {culprit}!")
231     if evidence_points > 10:
232         print("Ты собрал все улики и полностью доказал его вину. Отличная работа!")
233     else:
234         print("Ты раскрыл дело, но некоторых доказательств не хватало.")
235 else:
236     slow_print(f"Ошибка. Настоящий преступник – {culprit}.")
237     if evidence_points > 10:
238         print("Ты почти всё понял, но ошибся в выборе.")
239     else:
240         print("Твоё расследование провалилось. Попробуй снова!")
241
```

Рисунок 10 - Уровень 6. Обвинение

8. Итог расследования.

Итог выводит количество найденных улик, общее количество очков доказательств и полный список всех улик. `len(found_clues)` - считает количество найденных улик. `print(«», clue)` - выводит каждую улику с тире перед ней.

```
243     time.sleep(2)
244     print("\n=== ИТОГ РАССЛЕДОВАНИЯ ===")
245     print(f"Собрано улик: {len(found_clues)}")
246     print(f"Очки доказательств: {evidence_points}")
247     print("Все улики:")
248     for clue in found_clues:
249         print("-", clue)
250
251     slow_print("\nСпасибо за игру, детектив!")
252     slow_print("Удачи в дальнейших расследованиях!")
```

Рисунок 11 - Итог расследования

Вывод: Удалось разработать текстовую игру, используя основы программирования на Python, с упором на работу с коллекциями данных (списки, кортежи, множества, словари).